

Тема 12. Firestore. Работа с данными из JavaScript.



Хекслет колледж

Цель занятия:

Научить студентов выполнять все основные операции с Firestore из JavaScript: создавать, читать, обновлять и удалять данные (CRUD), а также делать простые запросы.

Учебные вопросы:

1. Основные CRUD операции в firestore
2. Простые запросы (фильтрация данных)

1. Основные CRUD операции в firestore

Что такое CRUD?

CRUD — это 4 базовые операции, которые можно делать с любыми данными в любом приложении:

- Create — Создать (добавить новые данные)
- Read — Прочитать (получить данные)
- Update — Обновить (изменить существующие данные)
- Delete — Удалить (убрать ненужные данные)

Это основа ЛЮБОГО приложения

Как работать с коллекциями и документами в Firestore

Метод **db.collection()** — работа с коллекциями

Что делает?

- Получает ссылку на коллекцию (папку). Это НЕ данные, а "адрес" коллекции.

Что принимает?

- Название коллекции (строка)

db.collection('products')

Что возвращает?

- `CollectionReference` — объект-ссылка на коллекцию.

Пример работы с коллекцией:

// Получаем ссылку на коллекцию

```
const productsRef = db.collection("products")
```

// Получаем все документы коллекции

```
const snapshot = await productsRef.get()
```

// Выводим данные

```
snapshot.forEach(doc => console.log(doc.id));  
snapshot.forEach(doc => console.log(doc.data()));
```

// Добавляем данные

```
const newProduct = {  
  name: "Samsung",  
  category: "smartphones",  
  price: 199000  
}
```

```
await productsRef.add(newProduct);
```

// Слушаем изменения всей коллекции

```
const unsubscribe = productsRef.onSnapshot((snapshot) => {  
  console.log("Данные изменились!");  
  snapshot.forEach((doc) => console.log(doc.data()));  
});
```

Метод `db.collection().doc()` — работа с документами

Что делает?

- Получает ссылку на конкретный документ (файл) в коллекции.

Что принимает?

- ID документа (строка)

`db.collection('books').doc('book123')` // файл book123

Что возвращает?

- `DocumentReference` — объект-ссылка на документ.

`const bookRef = db.collection('books').doc('book123')`

Важно!

- `DocumentReference` $\neq$ данные документа!
- Это только "адрес", где документ находится или будет находиться.

Пример:

```
const docRef = db.collection("products").doc("6pSjcsO6qqoit4QrEEBn");
```

```
// Получить данные документа
```

```
const snapshot = await docRef.get();
```

```
if (snapshot.exists) {
```

```
  const data = snapshot.data();
```

```
  console.log(data.name);
```

```
}
```

```
// Создать/перезаписать документ
```

```
await docRef.set({ name: "iPhone 17", price: "499999" })
```

// Обновить документ (частично)

```
await docRef.update({ price: "540000" });
```

// Удалить документ

```
await docRef.delete()
```

// Слушать изменения документа

```
const unsubscribe = docRef.onSnapshot((snapshot) => {  
  if (snapshot.exists) {  
    console.log("Данные документа изменились!");  
    console.log(snapshot.data());  
  }  
});
```

Reference $\neq$ Data

Как устроена работа с данными в Cloud Firestore из JavaScript



Firebase
Cloud Firestore

Надёжные
данные
для ваших
приложений

1 CollectionReference



Ссылка
на коллекцию

```
db.collection('books')
```

JS

- Это адрес коллекции, а не данные
- Можно выполнять:

`.get()` `.add()` `.where()`

`.onSnapshot()`

2 DocumentReference



Ссылка
на конкретный документ

```
db.collection('books').doc('book123')
```

JS

- Это адрес документа, а не его содержимое
- Можно выполнять:

`.get()` `.set()` `.update()`

`.delete()` `.onSnapshot()`

3 DocumentSnapshot / QuerySnapshot



Snapshot — это
результат чтения

```
// Для документа
const snapshot = await db
  .collection('books')
  .doc('book123').get();
```

JS

```
// Для коллекции
const snapshot = await db
  .collection('books').get();
```

- Для документа: DocumentSnapshot
- Для коллекции: QuerySnapshot
- Здесь уже можно проверить `exists` / `size`

4 Data



Вот это уже
реальные данные

```
const data = snapshot.data();
```

JS

```
{
  title: 'JavaScript',
  author: 'Иван',
  year: 2023
}
```

- Обычный JavaScript-объект
- С ним можно работать в коде



Запомнить: Reference — это адрес. Data — это содержимое.

Сначала получаем ссылку → потом читаем данные.



НЕ путать:

CollectionReference / DocumentReference	= где лежат данные
Snapshot	= результат чтения
data()	= сами данные

CRUD в Firestore

Четыре основные операции с данными в JavaScript



Firebase
Cloud Firestore

Надёжные
данные
для ваших
приложений

1 CREATE — Создание



- Добавляет новый документ
- `add()` — авто-ID
- `set()` — свой ID

```
await db.collection('books')  
  .add({  
    title: 'JS',  
    year: 2026  
  });
```

```
await db.collection('books')  
  .doc('book123')  
  .set({  
    title: 'JS',  
    year: 2026  
  });
```

Новый документ

2 READ — Чтение



- Получает данные из Firestore
- `get()` — разовый запрос
- `onSnapshot()` — слушает изменения

```
const snapshot = await db  
  .collection('books')  
  .get();
```

```
db.collection('books')  
  .onSnapshot(snapshot => {  
    // Обработка изменений  
    console.log(snapshot.docs);  
  });
```

Разово или в реальном времени

3 UPDATE — Обновление



- Меняет существующий документ
- `update()` — только указанные поля
- `FieldValue.increment()` / `arrayUnion()`

```
await db.collection('books')  
  .doc('book123')  
  .update({  
    year: 2027  
  });
```

```
{  
  views: firebase.firestore  
    .FieldValue.increment(1)  
}
```

Документ должен существовать

4 DELETE — Удаление



- Удаляет документ полностью
- `delete()` — удалить документ
- `FieldValue.delete()` — удалить поле

```
await db.collection('books')  
  .doc('book123')  
  .delete();
```

```
{  
  oldField: firebase.firestore  
    .FieldValue.delete()  
}
```

Документ или отдельное поле



Запомнить:

CREATE

`add()`, `set()`

READ

`get()`, `onSnapshot()`

UPDATE

`update()`

DELETE

`delete()`

CRUD — это основа работы с данными в Firestore. Те же операции, что и в любой базе данных, но с простым и удобным API.

Жизненный цикл документа



Создаём
(CREATE)



Читаем
(READ)



Обновляем
(UPDATE)



Удаляем
(DELETE)

Больше, чем просто хранение данных

CRUD - операции

1. CREATE — Создание данных **add()** — создать с авто-ID

// Что делает: Создает новый документ с автоматическим ID

// Что принимает: Объект с данными

// Что возвращает: Promise с DocumentReference нового документа

```
const newDocRef = await db.collection('books').add({  
  title: "JavaScript",  
  author: "Иван",  
  year: 2023  
});
```

```
console.log("Создан документ с ID:", newDocRef.id); // Например:  
"fH3kLm9pQrS"
```

set() — создать/перезаписать с указанным ID

// Что делает: Создает или полностью перезаписывает документ

// Что принимает: ID документа + объект с данными

// Что возвращает: Promise (успех/ошибка)

```
await db.collection('books').doc('my-book-id').set({  
  title: "React",  
  author: "Анна",  
  year: 2024  
});
```

// Вариант с merge (объединить, а не перезаписать):

```
await db.collection('books').doc('book123').set({  
  year: 2025 // Только это поле изменится  
}, { merge: true }); // Остальные поля сохранятся!
```

2. READ — Чтение данных

get() для коллекции — получить ВСЕ документы

Что делает:

- Получает все документы коллекции (разово)

Что возвращает:

- Promise с QuerySnapshot

```
const snapshot = await db.collection('books').get();
```



```
// Работа с результатом:
if (snapshot.empty) {
  console.log("Документов нет");
} else {
  console.log(` Нашлось ${snapshot.size} документов `);

  snapshot.forEach(doc => {
    console.log(doc.id);           // ID документа
    console.log(doc.data());       // Данные документа {title: "...", ...}
    console.log(doc.exists);       // Всегда true для коллекции
  });

  // Или как массив:
  const booksArray = snapshot.docs.map(doc => ({
    id: doc.id,
    ...doc.data()
  }));
}
```


get() для документа — получить ОДИН документ

Что делает:

- Получает один конкретный документ

Что возвращает:

- Promise с DocumentSnapshot

```
const snapshot = await db.collection('books').doc('book123').get();
```

```
if (snapshot.exists) {  
  const book = snapshot.data(); // {title: "...", author: "..."}  
  console.log(book.title);  
} else {  
  console.log("Документ не найден");  
}
```

onSnapshot() — слушать изменения в реальном времени

// ДЛЯ КОЛЛЕКЦИИ:

const unsubscribe =

db.collection('books').onSnapshot(snapshot => {

 // Выполнится: Сразу при подписке и при каждом
изменении

snapshot.forEach(doc => {

console.log(doc.data());

});

});

// ДЛЯ ДОКУМЕНТА:

```
const unsubscribe = db.collection('books').doc('book123').onSnapshot(
  snapshot => {
    if (snapshot.exists) {
      console.log("Новые данные:", snapshot.data());
    } else {
      console.log("Документ удален");
    }
  },
  error => {
    console.error("Ошибка:", error);
  }
);
```

// отписаться:

```
unsubscribe();
```

get() vs onSnapshot()

Два способа чтения данных из Cloud Firestore в JavaScript



Cloud Firestore

Гибкая база данных
для современных
приложений

Надёжные
данные
для ваших
приложений

1 get() — Разовое чтение

Читайте данные по запросу



- Один запрос → один ответ
- Возвращает данные только один раз
- Подходит для разовой загрузки
- Чтобы увидеть новые данные, нужно выполнить запрос снова

```
const snapshot = await db.collection('books').get();  
snapshot.forEach(doc => console.log(doc.data()));
```

JS



Возвращает:

Promise<QuerySnapshot>



Когда использовать:
загрузка списка, поиск,
кнопка "Обновить"

2 onSnapshot() — Подписка в реальном времени

Автоматически следите
за изменениями



- Подписка на изменения
- Срабатывает сразу после подписки
- Срабатывает при каждом изменении данных
- Интерфейс обновляется автоматически
- Нужно отписаться, когда слушатель больше не нужен

```
const unsubscribe = db.collection('books').onSnapshot(snapshot => {  
  snapshot.forEach(doc => console.log(doc.data()));  
});
```

JS



Возвращает:

функцию unsubscribe()



Когда использовать:
чат, корзина, уведомления,
live-список товаров



Как работает get()

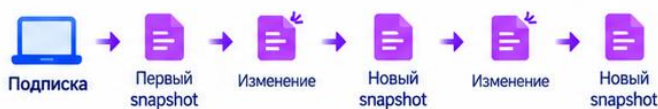


Ключевые различия

Критерий	get()	onSnapshot()
Сетевой режим	Разово	Постоянно слушает
Обновление UI	Вручную	Автоматически
Что вернуть	Promise<QuerySnapshot>	unsubscribe()



Как работает onSnapshot()



Запомнить: get() — когда данные нужны один раз. onSnapshot() — когда данные должны обновляться автоматически.



Оба метода можно применять
и к коллекции, и к документу.

3. UPDATE — Обновление данных

update() — частичное обновление

Что делает:

- Обновляет только указанные поля

Что принимает:

- Объект с полями для обновления

Что возвращает:

- Promise (успех/ошибка)

```
await db.collection('books').doc('book123').update({  
  year: 2025,  
  price: 1000,  
  // Можно обновлять вложенные поля:  
  "stats.views": 150,  
});
```

// Важно: Документ должен существовать! Иначе ошибка.

Специальные операторы обновления:

```
const FieldValue = firebase.firestore.FieldValue;  
await db.collection('books').doc('book123').update({
```

```
  // Увеличить числовое поле на 1  
  views: FieldValue.increment(1),
```

```
  // Добавить элемент в массив  
  tags: FieldValue.arrayUnion("js"),
```

```
  // Удалить поле из документа  
  oldField: FieldValue.delete(),
```

```
  // Записать текущее время сервера  
  updatedAt: FieldValue.serverTimestamp()  
});
```

Пример удаления элемента из массива:

```
await db.collection('books').doc('book123').update({  
  tags: FieldValue.arrayRemove("old")  
});
```

Запомните:

- `increment()` — изменить число
- `arrayUnion()` / `arrayRemove()` — изменить массив
- `delete()` — удалить поле
- `serverTimestamp()` — получить время сервера

add() / set() / update() – в чём разница

Три способа записи данных в Cloud Firestore из JavaScript



Надёжные
данные
для ваших
приложений

1

add()

Создать документ с авто-ID



- Создаёт новый документ в коллекции
- ID создаётся автоматически
- Принимает объект с данными
- Возвращает Promise с DocumentReference
- Подходит, когда ID заранее не важен

```
await db.collection('books').add({
  title: 'JavaScript',
  year: 2026
});
```



Новый документ + авто-ID

2

set()

Создать или полностью перезаписать документ



- Работает с конкретным document ID
- Может создать новый документ
- Может полностью заменить документ
- Принимает объект с данными
- Есть режим merge для частичного объединения

```
await db.collection('books').doc('book123').set({
  title: 'React',
  year: 2026
});
```

```
await db.collection('books').doc('book123').set({
  year: 2027
}, { merge: true });
```



Свой ID / можно merge

3

update()

Изменить только указанные поля



- Частично обновляет существующий документ
- Работает только с конкретным document ID
- Не перезаписывает весь документ
- Документ должен уже существовать
- Удобен для точечных изменений

```
await db.collection('books').doc('book123').update({
  year: 2027,
  price: 1000
});
```



Документ должен существовать

Критерий	add()	set()	update()
Создаёт новый документ	✓ Да	✓ Да	✗ Нет
Можно задать свой ID	✗ Нет	✓ Да	✓ Да
Автоматический ID	✓ Да	✗ Нет	✗ Нет
Полная перезапись документа	✗ Нет	✓ Да	✗ Нет
Частичное обновление	✗ Нет	– Только с merge	✓ Да
Документ должен существовать	✗ Нет	✗ Нет	✓ Да



Когда что использовать?



add()

Когда нужен новый документ и ID не важен



set()

Когда ID известен или нужно создать/заменить документ



update()

Когда нужно изменить отдельные поля существующего документа



Запомнить:

- add() = новый документ с авто-ID
- set() = создать или перезаписать документ
- update() = изменить существующий документ частично



Важно:

Если нужен частичный update через set(), используйте { merge: true }.

4. DELETE — Удаление данных

delete() — удалить документ

Что делает:

- Полностью удаляет документ

Что возвращает:

- Promise (успех/ошибка)

```
await db.collection('books').doc('book123').delete();  
console.log("Документ удален");
```

```
// Важно: После удаления document.exists будет false  
const snapshot = await db.collection('books').doc('book123').get();  
console.log(snapshot.exists()); // false
```

// Удаление поля в документе:

```
await db.collection('books').doc('book123').update({  
  oldField: firebase.firestore.FieldValue.delete()  
});
```

ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ

Получение ссылок (References)

db.collection() — ссылка на коллекцию

```
const booksRef = db.collection('books');
```

```
// booksRef.id = 'books'
```

```
// booksRef.path = 'books'
```

.doc() — ссылка на документ

```
const bookRef = db.collection('books').doc('book123');
```

```
// bookRef.id = 'book123'
```

```
// bookRef.path = 'books/book123'
```

```
// bookRef.parent = CollectionReference на 'books'
```

.ref — получение ссылки из snapshot

```
const snapshot = await db.collection('books').doc('book123').get();
```

```
const docRef = snapshot.ref; // DocumentReference на тот же документ
```

2. Простые запросы (фильтрация данных)

Базовые методы запросов:

where() — фильтрация по условию

// Операторы: ==, !=, <, <=, >, >=, array-contains, in, array-contains-any

```
const query = db.collection('books')
```

```
.where('year', '==', 2023)    // Год равен 2023
```

```
.where('price', '<', 1000)    // Цена меньше 1000
```

```
.where('tags', 'array-contains', 'js') // Массив содержит 'js'
```

```
.where('author', 'in', ['Иван', 'Анна']); // Автор Иван ИЛИ Анна
```

```
const snapshot = await query.get();
```

`orderBy()` — сортировка

```
const query = db.collection('books')  
  .orderBy('year', 'desc')    // Сначала новые (2024, 2023,  
  ...)  
  .orderBy('title', 'asc');   // Второй уровень сортировки
```

// Важно: Простые `where()` и `orderBy()` обычно работают на автоматических индексах. Для некоторых составных запросов требуется дополнительный `composite index`.

limit() — ограничение количества

// Только первые 10 книг

```
const query = db.collection('books')  
  .orderBy('year', 'desc')  
  .limit(10);
```

```
const snapshot = await query.get();  
console.log(snapshot.size); // максимум 10
```

startAt(), endAt(), startAfter(), endBefore() — пагинация
// Пагинация: пропустить первые 10, взять следующие 10

```
const firstPage = db.collection('books')  
  .orderBy('title')  
  .limit(10);
```

```
const snapshot = await firstPage.get();  
const lastDoc = snapshot.docs[snapshot.docs.length - 1];
```

```
const nextPage = db.collection('books')  
  .orderBy('title')  
  .startAfter(lastDoc)  
  .limit(10);
```

Как строится запрос Firestore

Пошаговая сборка запроса в JavaScript: от collection() до результата.

Гибкая работа с данными для ваших приложений

1 collection("books")



Выбираем коллекцию

С чего начинаем запрос.

```
JS
const query =
  db.collection('books');
```

2 where(...)



Фильтруем данные

Добавляем условия для отбора документов.

```
JS
.where('year', '==', 2023)
.where('price', '<', 1000)
```

3 orderBy(...)



Сортируем

Указываем поле и направление сортировки.

```
JS
.orderBy('year', 'desc')
```

4 limit(10)



Ограничиваем количество

Задаем, сколько документов вернуть.

```
JS
.limit(10)
```

5 get() / onSnapshot()



Выбираем способ чтения

Разовый запрос или подписка на изменения.

▶ **get()** — разовый запрос
Получает данные один раз.

⦿ **onSnapshot()** —
слушать изменения
Подписывается в реальном времени.

6 QuerySnapshot



Получаем результат

Работаем с полученными данными.

```
JS
snapshot.docs
snapshot.size
doc.data()
```

Полный пример запроса

```
const query = db.collection('books')
  .where('year', '==', 2023)
  .orderBy('title', 'asc')
  .limit(10);

const snapshot = await query.get();
```

JS

Сначала строим запрос (Query), добавляя нужные методы. Затем выполняем его с помощью get() или onSnapshot().



Формула запроса

Базовая последовательность методов:

collection → where → orderBy → limit → get / onSnapshot → snapshot



Что можно комбинировать

Часто используемые комбинации:

- ✓ where() + where() Несколько условий
- ✓ where() + orderBy() Фильтрация и сортировка
- ✓ orderBy() + limit() Сортировка и ограничение



Важно

- Некоторые сложные запросы требуют индекса.
- Firestore сам подскажет ссылку на создание индекса.
- Сначала строим Query, потом выполняем его.



Запомнить:

Запрос Firestore — это цепочка методов. Каждый следующий метод уточняет результат предыдущего.

Когда НУЖЕН составной индекс?

- Если существующих автоматических индексов недостаточно для compound query.
- Самый простой способ определить это: выполнить запрос → Firestore вернёт ошибку → перейти по ссылке → Create index.

Индексы Firestore: автоматический / составной

Какие индексы Cloud Firestore создает сам, а какие нужно создавать дополнительно



Firestore
Cloud Firestore

Надёжные
данные
для ваших
приложений

1

Автоматический индекс

Создается автоматически для простых запросов



Одинарные индексы создаются Firestore автоматически

- ✓ Создается Firestore автоматически
- ✓ Обычно работает для простых `where()`
- ✓ Обычно работает для обычного `orderBy()` по одному полю
- ✓ Поддерживает сравнения `=`, `<`, `<=`, `>`, `>=` по одному полю
- ✓ Для массивов есть поддержка `array-contains`
- ✓ Обычно ничего вручную делать не нужно

Пример простого запроса
(работает с автоматическим индексом)

```
const query = db.collection('books')
  .where('year', '==', 2023)
  .orderBy('year');
```

JS

2

Составной индекс (Composite Index)

Создается вручную для некоторых сложных запросов



Составные индексы создаются вручную при необходимости

- ✓ Создается вручную при необходимости
- ✓ Нужен для некоторых сложных запросов
- ✓ Часто встречается при комбинации `where()` + `orderBy()` по разным полям
- ✓ Может понадобиться для нескольких условий по разным полям
- ✓ Firestore сам подсказывает ссылку `Create index` при ошибке
- ! Если индекса не хватает — запрос вернет ошибку с подсказкой

Пример запроса, который может потребовать составной индекс

```
const query = db.collection('books')
  .where('year', '==', 2023)
  .orderBy('title')
  .limit(10);
```

JS



Запрос

Вы выполняете запрос к данным Firestore



Firestore проверяет индекс

Система проверяет, подходит ли существующий индекс для запроса



Индекс подходит → запрос выполняется

Данные возвращаются успешно



Индекса нет → ошибка + ссылка **Create index**

Запрос не выполняется. В ошибке будет ссылка для создания индекса.



Запомнить:

- 1 Простые запросы обычно используют автоматические индексы
- 2 Сложные составные запросы могут требовать composite index
- 3 Сначала запускаем запрос, затем при необходимости создаем индекс



Примеры

Простой запрос (автоматический индекс)

```
where('year', '==', 2023)
orderBy('year')
```

Составной запрос (может потребоваться индекс)

```
where('year', '==', 2023).orderBy('title')
where(...) + orderBy(...) + limit(...)
```

Как создать индекс?

Способ 1: По ссылке из ошибки (самый простой)

- Запустите запрос в коде.
- Получите ошибку в консоли браузера:

FirebaseError: The query requires an index.

You can create it here: <https://console.firebase.google.com/...>

- Перейдите по ссылке — она откроет Firebase Console
- Нажмите "Create index" — Firebase всё сделает за вас

Способ 2: Вручную в Firebase Console

Откройте Firebase Console

Выберите ваш проект

Слева: Firestore Database → Indexes

Нажмите "Create index"

Заполните поля:

- Collection ID: books (ваша коллекция)
- Fields to index:
- Field 1: year (тип: Ascending)
- Field 2: title (тип: Ascending)
- Query scope: Collection

Нажмите "Create" (создание займет 1-5 минут)

Разделение прав между обычными пользователями и администратором

3 основных подхода:

1. Custom Claims (самый правильный способ)

Что это?

Firebase позволяет добавлять "кастомные утверждения" (custom claims) к токену пользователя.

Это специальные поля, которые хранятся в токене JWT и проверяются на сервере.

Как работает:

В токене пользователя появляется:

```
{  
  uid: "user123",  
  email: "admin@site.com",  
  // ↓ Custom claims ↓  
  role: "admin",  
  permissions: ["create", "update", "delete"]  
}
```

Плюсы:

- Быстро (проверка на клиенте)
- Безопасно (можно проверить в правилах Firestore)
- Гибко (любые роли и права)

Минусы:

- Нужен Admin SDK (Node.js/Cloud Functions)
- Нельзя установить с фронтенда

2. Роль в Firestore

Как работает:

Храним роль пользователя в документе Firestore:

Коллекция users/{userId}:

```
{  
  email: "admin@site.com",  
  role: "admin", // или "user"  
  createdAt: "2026-01-15"  
}
```

Плюсы:

- Просто реализовать
- Можно делать с фронтенда
- Подходит для лабораторной

Минусы:

- Менее безопасно (пользователь может изменить свои данные)
- Нужно дополнительно проверять

3. Проверка по email (самый простейший)

Как работает:

Просто проверяем email пользователя:

```
const user = firebase.auth().currentUser;  
if (user.email === "admin@site.com") {  
  // Показать админ-панель  
}
```

Плюсы:

- Супер просто
- 5 минут на реализацию

Минусы:

- небезопасно (email можно подделать)
- негибко (только один админ)

Выводы по теме:

Операции CRUD:

- Create → `.add()`, `.set()`
- Read → `.get()`, `.onSnapshot()`
- Update → `.update()`
- Delete → `.delete()`

Сначала ссылка, потом данные:

// 1. Ссылка (где данные?)

```
const ref = db.collection('books').doc('id')
```

// 2. Данные (что там?)

```
const data = (await ref.get()).data()
```

Два типа чтения:

- Разовое → `.get()` (один запрос)
- Постоянное → `.onSnapshot()` (автообновление)

Контрольные вопросы:

- Что такое CRUD?
- Как создать документ с авто-ID?
- Как создать документ со своим ID?
- Как получить ВСЕ документы коллекции?
- Как получить ОДИН документ?
- Чем `.get()` отличается от `.onSnapshot()`?
- Как обновить документ?
- Как удалить документ?
- Что такое "ссылка" (Reference)?
- Как проверить, существует ли документ?
- Как фильтровать данные?
- Как сортировать данные?
- Что возвращает `.get()`?
- Как преобразовать `snapshot` в массив?
- Как слушать изменения документа?

хекслет колледж

@HEXLY.KZ